

High-Level Design (HLD) Centralized Gym Management System

Senior Database Architect & Full-Stack Tech Lead

February 22, 2026

Contents

1	Executive Summary	3
2	Module 1: Acquisition & Onboarding	3
2.1	Feature 1: Member Registration & Payment (100% Self-Service)	3
2.2	Feature 2: Trainer Recruitment Pipeline	3
2.3	Feature 3: Account Recovery (Forgot Password)	4
3	Module 2: Access Control & Daily Operations	4
3.1	Feature 4: Automated Check-In (The Digital Bouncer)	4
3.2	Feature 5: The Staff Live Monitor & Floor Operations	4
4	Module 3: Scheduling & Capacity Management	4
4.1	Feature 6: Master Schedule Creation	4
4.2	Feature 7: Member Class Booking	5
4.3	Feature 8: Trainer Class Operations & Client Logging	5
5	Module 4: Facility Management	5
5.1	Feature 9: Equipment Maintenance Tracking	5
6	Module 5: Administration & Analytics	6
6.1	Feature 10: The Admin Command Center	6
7	Role-Based Feature Breakdown & Sequence of Events	7
7.1	Entity: Member	7
7.1.1	1. The Digital Access Pass (Home Screen)	7
7.1.2	2. Subscription & Billing Hub	7
7.1.3	3. Class Discovery & Booking Calendar	7
7.1.4	4. Visit History & Progress Tracking	8
7.1.5	5. Profile & Settings Management	8
7.2	Entity: Trainer	8
7.2.1	1. The "My Schedule" Hub	8
7.2.2	2. Roster & Live Attendance Management	8
7.2.3	3. Client Progress & Notes	9

7.2.4	4. Performance & Payroll Metrics	9
7.2.5	5. Availability Management	9
7.3	Entity: Staff / Frontdesk	9
7.3.1	1. The Live Access Monitor	9
7.3.2	2. Member Lookup & Customer Support (Read-Only)	10
7.3.3	3. Equipment & Facility Management	10
7.3.4	4. Task Management & Operations	10
7.4	Entity: Manager / Admin	10
7.4.1	1. Financial & Revenue Command Center	10
7.4.2	2. User & Role Management (RBAC) & Recruitment	11
7.4.3	3. Master Schedule & Facility Configuration	11
7.4.4	4. Advanced Analytics & Master Inventory	11

1 Executive Summary

This document outlines the architectural blueprint for a centralized, "lean" Gym Management System. Built on a 3-tier architecture (React Frontend, Spring Boot Backend, PostgreSQL Database), the system is designed to eliminate revenue leakage, scheduling conflicts, and data redundancy by enforcing strict business logic at the database level and pushing data-entry responsibilities to a 100% self-service member model.

2 Module 1: Acquisition & Onboarding

2.1 Feature 1: Member Registration & Payment (100% Self-Service)

Goal: A frictionless sign-up process where the user does the data entry and handles their own secure payment.

1. **The Arrival:** A new user walks into the facility. Staff points them to a QR code on the front desk.
2. **Profile Creation:** The user scans the code, which opens the React web application on their mobile device. They enter their Name, Email, and create a secure Password.
3. **Checkout:** They select a membership tier (e.g., "VIP Monthly") and pay via credit card using a secure integration (e.g., Stripe or Apple Pay).
4. **System Action:** The Spring Boot backend securely hashes the password using BCrypt, saves the user in the `users` table, logs the payment in the `invoices_payments` table, and triggers PostgreSQL to instantly mark their `subscriptions` status as `ACTIVE`.
5. **The Result:** The user instantly receives a digital barcode on their screen. They are fully onboarded in under two minutes without staff intervention.

2.2 Feature 2: Trainer Recruitment Pipeline

Goal: Streamline hiring without cluttering the main user database with rejected applicants.

1. **The Application:** A prospective trainer visits the public "Careers" page, fills out their contact details, and uploads their fitness certifications.
2. **System Action (Staging):** This data is saved in a separate `trainer_applications` table with a default status of `PENDING`.
3. **Admin Review:** The Admin views a Kanban-style recruitment board on their dashboard, reviews the applicant's credentials, conducts an interview, and clicks "Approve & Hire".
4. **System Action (Migration):** Spring Boot automatically migrates the applicant's data from `trainer_applications` into the main `users` table, assigns them the `TRAINER` role, and sends an automated email linking them to set up their password.

2.3 Feature 3: Account Recovery (Forgot Password)

Goal: Zero staff intervention for locked-out accounts.

1. **The Request:** A user forgets their password and clicks "Forgot Password" on the mobile or web login screen.
2. **System Action:** Spring Boot generates a secure, time-sensitive token and emails a password reset link directly to the user, bypassing the front desk entirely.

3 Module 2: Access Control & Daily Operations

3.1 Feature 4: Automated Check-In (The Digital Bouncer)

Goal: Fast, secure entry where PostgreSQL enforces the rules and Staff simply monitors.

1. **The Scan:** An active Member scans their digital barcode at the front turnstile or scanner pad.
2. **System Action:** The scanner hits the Spring Boot API, which calls a PostgreSQL Stored Procedure (`sp.ProcessFrontDeskCheckIn`). In milliseconds, the database verifies if the subscription is **ACTIVE** and not past its expiration date.
3. **The Verdict (Valid):** If valid, the turnstile opens. The Staff's iPad flashes **GREEN** with the member's photo, and the staff welcomes them in.
4. **The Verdict (Invalid):** If unpaid or expired, the turnstile remains locked. The Staff's iPad flashes **RED**, and staff directs the member to update their payment method on their app.

3.2 Feature 5: The Staff Live Monitor & Floor Operations

Goal: Repurpose front desk clerks into active floor managers and community hosts.

1. **Live Feed:** Staff monitor a real-time WebSocket feed on an iPad showing check-ins as they happen, allowing them to intercept issues immediately.
2. **Member Support:** Staff use a Read-Only search function to look up member profiles to answer questions (e.g., locker assignments, upcoming booked classes).
3. **Manual Override:** In edge cases (e.g., a member's phone battery dies), Staff can visually verify the member's identity against their database photo and click "Override" to manually unlock the turnstile.

4 Module 3: Scheduling & Capacity Management

4.1 Feature 6: Master Schedule Creation

Goal: The Admin builds the gym's schedule without ever double-booking a room or a trainer.

1. **The Setup:** The Admin attempts to assign "Yoga" to Studio A at 6 PM with Trainer Bob, setting a maximum capacity of 20 members.
2. **System Action:** Before the save commits, a PostgreSQL trigger (`trg_PreventDoubleBooking`) scans the `classes` table. If Studio A is already in use, or if Trainer Bob is scheduled to teach elsewhere at 6 PM, the database explicitly rejects the transaction, preventing human error.

4.2 Feature 7: Member Class Booking

Goal: Members manage their own schedules; classes never overfill.

1. **The Booking:** A Member opens their app, navigates to the calendar, sees the 6 PM Yoga class, and taps "Book".
2. **System Action:** Spring Boot initiates a transaction. A PostgreSQL trigger verifies the member has an active plan and checks that the class capacity has not reached its limit (e.g., 20). If valid, the `bookings` record is inserted.
3. **The Waitlist:** If the class is full, the system rejects the booking and offers the member a spot on the waitlist. If a booked member cancels, the system auto-promotes the first waitlisted member.

4.3 Feature 8: Trainer Class Operations & Client Logging

Goal: Give trainers the tools to run their classes and train clients without seeing the gym's financial data.

1. **Schedule View:** The Trainer logs into their portal and sees a filtered view of only their assigned classes.
2. **Live Roster:** The Trainer opens the 6 PM Yoga class, views the exact roster of 20 booked members, and taps "Present" or "No-Show" to take attendance.
3. **PT Logging:** For 1-on-1 personal training clients, the Trainer accesses the client's profile to log workout metrics (weights, reps) and private notes directly into the `client_progress_logs` table.

5 Module 4: Facility Management

5.1 Feature 9: Equipment Maintenance Tracking

Goal: Keep the gym floor safe and eliminate paper "Out of Order" signs.

1. **The Discovery:** A Staff member or Trainer notices a malfunctioning piece of equipment (e.g., a broken treadmill cable).
2. **The Log:** They open their internal app, select the specific machine, and update its status in the `equipment` table from `AVAILABLE` to `NEEDS MAINTENANCE`, adding a brief note.

3. **The Resolution:** The Admin sees this alert on their command center dashboard, contacts the repair vendor, and updates the status back to **AVAILABLE** once the machine is fixed.

6 Module 5: Administration & Analytics

6.1 Feature 10: The Admin Command Center

Goal: The owner oversees the financial and operational health of the business using perfectly accurate, database-enforced metrics.

1. **Financial Reporting:** Because all transactions are processed through the self-service portal, the database perfectly mirrors bank deposits. The Admin views dashboards powered by complex SQL aggregations showing Monthly Recurring Revenue (MRR) and failed transaction rates.
2. **Utilization Metrics:** The Admin views reports on Peak Gym Hours (to optimize cleaning/staff schedules) and Trainer Utilization (to identify the most and least popular classes).
3. **Configuration Management:** The Admin executes high-level system changes, such as updating membership plan pricing, adding new room layouts to the **facilities** table, and managing global user roles (hiring/firing).

7 Role-Based Feature Breakdown & Sequence of Events

This section details the exact dashboard features, user interface views, and backend sequence of events categorized by the four primary system entities.

7.1 Entity: Member

Primary Role: Self-service consumer. Responsible for their own data entry, bookings, and payments.

7.1.1 1. The Digital Access Pass (Home Screen)

- **Goal:** Get the member into the gym with zero friction.
- **What they see:** A dynamic QR Code/Barcode, their status ("ACTIVE" in green), and a quick scan of their upcoming booked class.
- **Sequence / Actions:** Member holds their phone up to the scanner at the front desk or turnstile.
- **Behind the Scenes:** The barcode safely encrypts their `user_id`. When scanned, it triggers the `sp_ProcessFrontDeskCheckIn` stored procedure to verify entry permissions.

7.1.2 2. Subscription & Billing Hub

- **Goal:** Handle all revenue collection automatically.
- **What they see:** Breakdown of their current plan, next billing date, and payment method on file.
- **Sequence / Actions:** Member can securely type in a new credit card, upgrade/downgrade their plan, or download PDF receipts.
- **Behind the Scenes:** Pulls data from `subscriptions` and `invoices_payments`. Failed payments automatically flip the subscription to "INACTIVE", turning their access barcode red.

7.1.3 3. Class Discovery & Booking Calendar

- **Goal:** Seamless reservation of group fitness spots.
- **What they see:** Weekly calendar showing available classes, trainers, and remaining spots.
- **Sequence / Actions:** Member taps "Book" to reserve, joins a waitlist if full, or cancels an existing booking.
- **Behind the Scenes:** Spring Boot attempts an INSERT into the `bookings` table. A PostgreSQL trigger (`trg_MaintainClassCapacity`) instantly verifies capacity limits before committing the transaction.

7.1.4 4. Visit History & Progress Tracking

- **Goal:** Keep the member motivated to reduce churn.
- **What they see:** A log of their fitness journey.
- **Sequence / Actions:** View a list of recent check-ins and read workout logs/notes left by their Personal Trainer.

7.1.5 5. Profile & Settings Management

- **Goal:** Maintain accurate database records without staff intervention.
- **What they see:** Standard profile settings page.
- **Sequence / Actions:** Member updates their phone, email, address, or password.
- **Behind the Scenes:** Triggers an UPDATE query to the `users` table.

7.2 Entity: Trainer

Primary Role: Service provider. Focused on teaching, client tracking, and schedule management without access to financial data.

7.2.1 1. The "My Schedule" Hub

- **Goal:** Show trainers exactly where they need to be and when.
- **What they see:** A mobile-friendly calendar showing only their assigned group classes or 1-on-1 sessions.
- **Sequence / Actions:** Click a class to see room assignments, times, and real-time booking capacity.
- **Behind the Scenes:** Spring Boot runs a SELECT query on the `classes` table, strictly filtering by their `trainer_id`.

7.2.2 2. Roster & Live Attendance Management

- **Goal:** Real-time visibility into class attendance.
- **What they see:** A list of names and photos of successfully booked members.
- **Sequence / Actions:** Trainer taps "Present" or "No-Show" on their tablet. They can also manually admit waitlisted members if a booked member fails to show.
- **Behind the Scenes:** Updates the `bookings` table, providing data for admins to track class delinquency.

7.2.3 3. Client Progress & Notes

- **Goal:** Track fitness journeys of dedicated 1-on-1 clients.
- **What they see:** Searchable directory of their personal training clients.
- **Sequence / Actions:** Trainer logs workout metrics (weights, reps) and adds private internal notes (e.g., injury accommodations).
- **Behind the Scenes:** Inserts data into the `client_progress_logs` table, linking `trainer_id`, `member_id`, and a JSON/text payload.

7.2.4 4. Performance & Payroll Metrics

- **Goal:** Transparency regarding work and compensation.
- **What they see:** Mini-analytics dashboard for the current month.
- **Sequence / Actions:** View personal stats like total hours taught and class fill-rate percentage.

7.2.5 5. Availability Management

- **Goal:** Prevent Admins from scheduling trainers when unavailable.
- **What they see:** Settings page for working hours.
- **Sequence / Actions:** Trainer blocks out vacation, sick days, or unavailable slots.
- **Behind the Scenes:** PostgreSQL trigger (`trg_PreventDoubleBooking`) reads this data and rejects Admin attempts to schedule classes during these blocked periods.

7.3 Entity: Staff / Frontdesk

Primary Role: Floor operations and community hosts. No data entry or financial modification privileges.

7.3.1 1. The Live Access Monitor

- **Goal:** Keep the building secure and assist with tech issues.
- **What they see:** Real-time scrolling feed of members scanning in.
- **Sequence / Actions:** Monitor screen for Green (Granted) or Red (Denied). If a member's phone dies, staff can visually verify their face and click "Override - Let In".
- **Behind the Scenes:** React frontend uses WebSockets via Spring Boot to listen to the `check_ins` table for instant updates.

7.3.2 2. Member Lookup & Customer Support (Read-Only)

- **Goal:** Answer member questions without modifying accounting data.
- **What they see:** Search bar for member names/phone numbers.
- **Sequence / Actions:** View upcoming booked classes or locker assignments. Send automated password reset links. They cannot view credit cards or alter pricing.

7.3.3 3. Equipment & Facility Management

- **Goal:** Maintain safe, functional gym floors.
- **What they see:** Visual list/map of gym equipment.
- **Sequence / Actions:** Log broken machines (e.g., snapped cable) as **NEEDS MAINTENANCE**, alerting the Admin. Update retail inventory counts.
- **Behind the Scenes:** Interacts directly with the **equipment** table to update status and timestamps.

7.3.4 4. Task Management & Operations

- **Goal:** Ensure facility cleanliness and readiness.
- **What they see:** Shift-based checklists (Opening, Mid-Day, Closing).
- **Sequence / Actions:** Check off physical tasks (e.g., "Sanitized mats").
- **Behind the Scenes:** Logs completed tasks in a **daily_tasks** table for Admin accountability reviews.

7.4 Entity: Manager / Admin

Primary Role: Business owner/operator. God-mode access to financial health, global configuration, and personnel.

7.4.1 1. Financial & Revenue Command Center

- **Goal:** Track revenue and manage core pricing.
- **What they see:** High-level view of MRR (Monthly Recurring Revenue) and transaction success rates.
- **Sequence / Actions:** Create/Edit membership plans, adjust pricing, and issue strict refunds.
- **Behind the Scenes:** Powered by complex SQL aggregations summing revenue by month and plan type from the **invoices_payments** table.

7.4.2 2. User & Role Management (RBAC) & Recruitment

- **Goal:** Control system access and manage hiring.
- **What they see:** Master user directory and a Kanban board for trainer recruitment.
- **Sequence / Actions:** Hire new applicants by clicking "Approve", which auto-migrates data to the `users` table. Fire staff by revoking system access. Ban problematic members.

7.4.3 3. Master Schedule & Facility Configuration

- **Goal:** Define physical layout and macro-schedule.
- **What they see:** Macro-view calendar and facility list.
- **Sequence / Actions:** Add new rooms/studios and assign trainers to classes.
- **Behind the Scenes:** Protected by database constraints. Double-booking a room or trainer triggers a hard rejection from PostgreSQL.

7.4.4 4. Advanced Analytics & Master Inventory

- **Goal:** Data-driven business decisions and capital oversight.
- **What they see:** Charts mapping peak gym hours, trainer utilization, churn rates, and a master list of equipment.
- **Sequence / Actions:** Use check-in data to optimize cleaning schedules. Dispatch repair vendors based on Staff equipment flags. Retire persistently broken machines from the database.